

Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
ΔΠΜΣ Επιστήμη Δεδομένων και Μηχανική Μάθησης
Διαχείριση Δεδομένων Μεγάλης Κλίμακας

Εξαμηνιαία Εργασία

Αναστάσιος Κανέλλος
ΑΜ: 03400251

29 Ιουνίου 2026

Περιεχόμενα

1	Μορφές Αποθήκευσης Δεδομένων	2
1.1	Αξιολόγηση CSV και Εναλλακτικές Μορφές	2
1.2	Επιλογή Εναλλακτικού Format: Parquet	2
1.3	Μετατροπή και Σύγκριση Επίδοσης	2
2	Query 1 - DataFrame και RDD API	3
2.1	Υλοποίηση DataFrame χωρίς UDF - DFQ1.py	3
2.2	Υλοποίηση DataFrame με UDF - DFQ1_udf.py	4
2.3	Υλοποίηση RDD - RddQ1.py	4
2.4	Σύγκριση Επίδοσης (2 executors × 1 core × 2 GB)	5
3	Query 2 - DataFrame και SQL API	6
3.1	Υλοποίηση DataFrame - DFQ2.py	6
3.2	Υλοποίηση SQL - SQLQ2.py	6
3.3	Σύγκριση Επίδοσης (4 executors × 1 core × 2 GB)	7
4	Query 3 - DataFrame και RDD API	7
4.1	Υλοποίηση DataFrame - DFQ3.py	7
4.2	Υλοποίηση RDD - RddQ3.py	8
4.3	Σύγκριση Επίδοσης (3 executors × 1 core × 2 GB)	10
5	Query 4 - Cross Join and Scalability	10
5.1	Υλοποίηση DataFrame - DFQ4.py	10
5.2	Στρατηγικές Join από τον Optimizer	11
5.3	Μελέτη Κλιμακωσιμότητας	11
6	Join Strategies - Hint & Explain	12
6.1	Query 3 - DataFrame API	12
6.2	Query 4	12
6.3	Σύγκριση Στρατηγικών	13

1 Μορφές Αποθήκευσης Δεδομένων

1.1 Αξιολόγηση CSV και Εναλλακτικές Μορφές

Η μορφή **CSV** δεν είναι ιδανική για επεξεργασία με Spark¹, για τους εξής λόγους:

- **Row Storage**: κάθε ανάγνωση φέρνει ολόκληρη τη γραμμή ακόμη κι αν χρειάζονται λίγες στήλες. Οδηγεί σε σπατάλη I/O σε queries που αγγίζουν υποσύνολο στηλών.
- **Απουσία schema**: απαιτείται schema inference ή ρητή δήλωση τύπων. Οδηγεί σε επιπλέον σάρωση των δεδομένων και casts κατά το runtime.
- **Δεν υποστηρίζει predicate pushdown**: δεν υπάρχουν στατιστικές ή ευρετήρια· τα φίλτρα εφαρμόζονται αφού διαβαστούν όλα τα δεδομένα, χωρίς να παραλείπεται I/O.
- **Μη splittable όταν συμπιέζεται (π.χ. gzip)**: ένα gzip-CSV αρχείο διαβάζεται από έναν μόνο task, εξαλείφοντας τον παραλληλισμό.

Οι κυριότερες εναλλακτικές μορφές αποθήκευσης για Spark συνοψίζονται στον Πίνακα 1.

Πίνακας 1: Εναλλακτικές μορφές αποθήκευσης για χρήση με Apache Spark

Μορφή	Αποθήκευση	Schema	Pred. Pushdown
Parquet	στήλη	ναι	ναι
ORC	στήλη	ναι	ναι
Avro	γραμμή	ναι	μερικό ²

Στα Parquet και ORC τα δεδομένα αποθηκεύονται **ανά στήλη**, οπότε τιμές ίδιου τύπου και συχνά παρόμοιες βρίσκονται μαζί· αυτή η ομοιογένεια επιτρέπει απλά encodings (π.χ. αντικατάσταση επαναλαμβανόμενων τιμών με σύντομους κωδικούς) που πετυχαίνουν υψηλό βαθμό συμπίεσης, εφαρμοσμένες ανά block³ ώστε το αρχείο να παραμένει splittable. Το Avro, ως μορφή **ανά γραμμή**, συμπιέζει ανά block εγγραφών αλλά χωρίς το όφελος της ομαδοποίησης ανά στήλη.

1.2 Επιλογή Εναλλακτικού Format: Parquet

Επιλέχθηκε η μορφή **Parquet** για τους εξής λόγους:

- **Column pruning**: τα queries που προσπελαίνουν υποσύνολο στηλών (π.χ. μόνο Premis Desc, TIME OCC) διαβάζουν μόνο τις αναγκαίες στήλες.
- **Predicate pushdown**: φίλτρα εκτελούνται κατά την ανάγνωση, μειώνοντας το I/O.
- **Ενσωματωμένο schema**: δεν απαιτείται inference.

1.3 Μετατροπή και Σύγκριση Επίδοσης

Η μετατροπή CSV→Parquet εκτελείται μία φορά σε ξεχωριστό spark-submit job (convert_to_parquet.py) και το αποτέλεσμα αποθηκεύεται μόνιμα στο HDFS. Στη συνέχεια, σε κάθε εκτέλεση το Parquet φορτώνεται από το HDFS ακριβώς, όπως τα αρχικά CSV αρχεία. Η σύγκριση γίνεται τρέχοντας το ίδιο query με όρισμα --format csv και --format parquet.

¹<https://motherduck.com/learn/why-choose-parquet-table-file-format/>

²Το Avro είναι row-based και δεν κρατάει στατιστικά (min/max) ανά στήλη, οπότε δεν μπορεί να παραλείψει blocks με βάση την τιμή ενός φίλτρου (I/O skipping), όπως τα Parquet/ORC. Επιτρέπει μόνο περιορισμένες βελτιστοποιήσεις (π.χ. ανάγνωση υποσυνόλου πεδίων), για αυτό και «μερικό».

³Με block εννοούμε ένα αυτοτελές τμήμα του αρχείου (row-group στο Parquet, stripe στο ORC) που περιέχει ένα σύνολο γραμμών, συμπιέζεται ανεξάρτητα και φέρει τα δικά του στατιστικά. Αυτή η ανεξαρτησία επιτρέπει σε διαφορετικά tasks να διαβάζουν διαφορετικά blocks παράλληλα.

```

1 # convert_to_parquet.py -- one-time conversion (separate job)
2 df = spark.read.csv([CRIME_2010, CRIME_2020], header=True, inferSchema=False
3 )
4 df.write.mode("overwrite").parquet(f"{base_path}/LA_Crime_Data.parquet")
5 # DFQ1.py -- pick data source via --format {csv,parquet}
6 if args.format == "parquet":
7     crimes_df = spark.read.parquet(f"{base_path}/LA_Crime_Data.parquet")
8 else:
9     crimes_df = spark.read.csv([CRIME_2010, CRIME_2020], header=True,
10 inferSchema=False)

```

Κώδικας 1: convert_to_parquet.py και επιλογή μορφής στο DFQ1.py

Το Query 1 προσπελαίνει μικρό υποσύνολο στηλών, οπότε η **column pruning** του Parquet μειώνει σημαντικά το I/O, ενώ το **predicate pushdown** για το φίλτρο `Premis Desc = "STREET"` μειώνει τους rows που επεξεργάζεται ο Spark.

Πίνακας 2: Χρόνοι εκτέλεσης Query 1 (DataFrame χωρίς UDF): CSV vs Parquet - 2 executors × 1 core × 2 GB

Μορφή	Run 1 (s)	Run 2 (s)	Run 3 (s)	Median (s)
CSV	51.507	44.596	36.807	44.596
Parquet	28.568	24.966	23.368	24.966

Σχολιασμός: Το Parquet είναι σταθερά ταχύτερο (median 25.0s έναντι 44.6s). Αυτό οφείλεται στο column pruning και το predicate pushdown που εφαρμόζονται κατά την ανάγνωση από το HDFS: ο Spark φορτώνει μόνο τις αναγκαίες στήλες (`Premis Desc`, `TIME OCC`) και παραλείπει row groups που δεν πληρούν το φίλτρο `Premis Desc = "STREET"`, μειώνοντας το I/O. Το πλεονέκτημα αυτό υλοποιείται **μόνο κατά την αρχική σάρωση** που γεμίζει την cache (κατά την `count()`). Η επόμενη action (`show()`) ανακτά τα δεδομένα από τη μνήμη ανεξαρτήτως μορφής αρχείου, οπότε σε αυτό το στάδιο το Parquet και τα πλεονεκτήματά του δεν προσφέρουν επιπλέον όφελος. Έτσι η επίδραση του format περιορίζεται στο κόστος της μίας αρχικής ανάγνωσης.

2 Query 1 - DataFrame και RDD API

Το Query 1 ταξινομεί τα τμήματα της ημέρας κατά φθίνουσα σειρά ανάλογα με το ποσοστό εγκλημάτων σε δρόμο (`Premis Desc = "STREET"`). Ορισμός τμημάτων: Πρωί 05:00–11:59, Απόγευμα 12:00–16:59, Βράδυ 17:00–20:59, Νύχτα 21:00–04:59.

2.1 Υλοποίηση DataFrame χωρίς UDF - DFQ1.py

Αρχικά διαβάζονται τα δύο CSV αρχεία σε ένα DataFrame και φιλτράρονται οι εγγραφές με `Premis Desc = "STREET"`. Στη συνέχεια, η στήλη `TIME OCC` μετατρέπεται σε ακέραιο και αντιστοιχίζεται σε τμήμα της ημέρας μέσω αλυσίδας built-in εκφράσεων `when()`, οι οποίες αποτιμώνται απευθείας από τον Catalyst χωρίς έξοδο από τη JVM. Το ενδιαμέσο DataFrame χρησιμοποιείται από δύο actions (την `count()` για το σύνολο των εγγραφών, καθώς και την `show()` για την εγγραφή του αποτελέσματος), οπότε αποθηκεύεται στη μνήμη με `cache()`: η πρώτη action πυροδοτεί τον υπολογισμό και γεμίζει την cache, ενώ η επόμενη το διαβάσει από εκεί και αποφεύγει νέα σάρωση των CSV. Έπειτα, εφαρμόζεται aggregation και υπολογίζεται το ποσοστό εγκλημάτων ανά τμήμα ημέρας, και ταξινόμηση κατά φθίνουσα σειρά.

```

1 crimes_df = spark.read.csv([CRIME_2010, CRIME_2020], header=True,
2 inferSchema=False)

```

```

2 street_df = crimes_df.filter(col("Premis Desc") == "STREET")
3
4 time_col = col("TIME OCC").cast("integer")
5 period_df = street_df.withColumn(
6     "day_period",
7     when((time_col >= 500) & (time_col <= 1159), "Morning")
8     .when((time_col >= 1200) & (time_col <= 1659), "Afternoon")
9     .when((time_col >= 1700) & (time_col <= 2059), "Evening")
10    .when((time_col >= 2100) | (time_col <= 459), "Night")
11 )
12
13 period_df.cache()           # cache before the two actions below
14
15 total = period_df.count()   # 1st action: triggers caching
16
17 result = (
18     period_df.groupBy("day_period")
19     .count()
20     .withColumn("percentage", round(col("count") / total * 100, 2))
21     .orderBy(col("percentage").desc())
22     .select("day_period", "percentage")
23 )
24 result.show()              # 2nd action: reads from cache

```

Κώδικας 2: DFQ1.py - Ταξινόμηση περιόδου με when()

2.2 Υλοποίηση DataFrame με UDF - DFQ1_udf.py

Η λογική εδώ είναι ταυτόσημη με την προηγούμενη (ανάγνωση CSV, φιλτράρισμα STREET, ομαδοποίηση και υπολογισμός ποσοστών). Διαφέρει μόνο ο τρόπος αντιστοίχισης της ώρας σε τμήμα ημέρας. Αντί για when(), ορίζεται μια συνάρτηση Python (classify_period) που υλοποιεί την ίδια λογική με συνθήκες if, καταχωρείται ως UDF και εφαρμόζεται γραμμή-γραμμή για τη δημιουργία της στήλης day_period. Όπως θα φανεί στη σύγκριση, η UDF είναι αδιαφανής (opaque) για τον optimizer και εισάγει κόστος επικοινωνίας JVM-Python.

```

1 def classify_period(time_occ: str):
2     if time_occ is None:
3         return None
4     t = int(time_occ)
5     if 500 <= t <= 1159: return "Morning"
6     if 1200 <= t <= 1659: return "Afternoon"
7     if 1700 <= t <= 2059: return "Evening"
8     if t >= 2100 or t <= 459: return "Night"
9
10 period_udf = udf(classify_period, StringType())
11 period_df = street_df.withColumn("day_period", period_udf(col("TIME OCC")))

```

Κώδικας 3: DFQ1_udf.py - Python UDF για ταξινόμηση χρονικής περιόδου

2.3 Υλοποίηση RDD - RddQ1.py

Πρώτα διαβάζεται η επικεφαλίδα του πρώτου αρχείου (first()) ώστε να εντοπιστούν δυναμικά οι δείκτες στηλών TIME OCC και Premis Desc με βάση το όνομά τους. Στη συνέχεια, και τα δύο αρχεία διαβάζονται ως ακατέργαστες γραμμές κειμένου (textFile) και ενώνονται σε ένα RDD (union). Εφαρμόζεται αλυσίδα transformations:

1. filter: αφαιρείται η γραμμή επικεφαλίδας ώστε να μην αντιμετωπιστεί ως εγγραφή.
2. map με csv.reader: κάθε γραμμή κειμένου αναλύεται σε λίστα πεδίων. Χρησιμοποιείται csv.reader αντί για split(","), καθώς μπορεί να υπάρχουν πεδία με κόμματα εντός εισαγωγικών.

3. filter: κρατούνται μόνο οι εγγραφές με Premis Desc = "STREET".

Το φιλτραρισμένο RDD αποθηκεύεται στη μνήμη με cache(), ώστε να μην επαναλαμβάνεται η σάρωση και ανάλυση των CSV για κάθε επόμενη action. Η πρώτη action (count()) μετράει το σύνολο των εγγραφών STREET και γεμίζει την cache. Η δεύτερη φάση διαβάζει απευθείας από τη μνήμη με αλυσίδα transformations:

1. map με classify_period: κάθε γραμμή αντικαθίσταται από ένα label τμήματος ημέρας ("Morning", "Night" κ.λπ.) ή None αν η ώρα δεν ταξινομείται.
2. filter: αφαιρούνται οι None τιμές.
3. map: κάθε label μετατρέπεται σε ζεύγος (period, 1).
4. reduceByKey: τα ζεύγη αθροίζονται ανά κλειδί, δηλαδή (period, count).
5. collect (action): τα αποτελέσματα επιστρέφονται στον driver.

```

1 # Resolve column indices dynamically from the header
2 header_line = sc.textFile(CRIME_2010).first()
3 header      = next(csv.reader(io.StringIO(header_line)))
4 time_idx    = header.index("TIME OCC")
5 premis_idx  = header.index("Premis Desc")
6
7 # Phase 1: build and cache the filtered RDD
8 street_rdd = (
9     sc.textFile(CRIME_2010).union(sc.textFile(CRIME_2020))
10    .filter(lambda line: line != header_line)           # drop header
11    .map(lambda line: next(csv.reader(                   # parse CSV row
12        io.StringIO(line))))                           # -> list of fields
13    .filter(lambda row: row[premis_idx] == "STREET")
14 )
15 street_rdd.cache()
16
17 total = street_rdd.count()    # 1st action: triggers caching
18
19 # Phase 2: reads from cache
20 period_counts = (
21     street_rdd
22     .map(lambda row: classify_period(row[time_idx]))    # field -> period
23     .filter(lambda p: p is not None)                   # drop unclassified
24     .map(lambda p: (p, 1))                             # -> (label, 1) pairs
25     .reduceByKey(lambda a, b: a + b)                  # count per period
26     .collect()                                         # 2nd action: to
27     driver
28 )

```

Κώδικας 4: RddQ1.py - RDD με clear map-reduce λογική

2.4 Σύγκριση Επίδοσης (2 executors × 1 core × 2 GB)

Πίνακας 3: Χρόνοι εκτέλεσης Query 1 - 3 εκτελέσεις, 2 executors × 1 core × 2 GB

Υλοποίηση	Run 1 (ς)	Run 2 (s)	Run 3 (s)	Median (s)
DataFrame (χωρίς UDF)	51.507	44.596	36.807	44.596
DataFrame (με UDF)	51.161	34.155	78.936	51.161
RDD	51.398	77.107	57.359	57.359

Σχολιασμός: Η κατάταξη των διαμέσων ακολουθεί τη θεωρητική αναμενόμενη σειρά:

DataFrame χωρίς UDF (44.6 s) < DataFrame με UDF (51.2 s) < RDD (57.4 s)

Το DataFrame χωρίς UDF είναι ταχύτερο διότι οι built-in συναρτήσεις (`when()`) εκτελούνται εξ ολοκλήρου από τον Spark εσωτερικά, χωρίς να χρειαστεί να «βγουν» στον Python κώδικα. Η εκδοχή με UDF είναι πιο αργή, διότι για κάθε γραμμή ο Spark πρέπει να στείλει τα δεδομένα στη Python διεργασία, να εκτελεστεί η συνάρτηση και να επιστρέψει το αποτέλεσμα· αυτή η «μεταφορά» ανά γραμμή εμπεριέχει κόστος. Το RDD είναι το πιο αργό καθώς ολόκληρη η επεξεργασία (φιλτράρισμα, ταξινόμηση, μέτρηση) γίνεται με custom/manual map-reduce σε Python, χωρίς καμία αυτόματη βελτιστοποίηση από τον Spark.

3 Query 2 - DataFrame και SQL API

Το Query 2 βρίσκει, ανά έτος, τους 3 μήνες με τον υψηλότερο αριθμό καταγεγραμμένων εγκλημάτων, με κατάταξη ανά έτος.

3.1 Υλοποίηση DataFrame - DFQ2.py

Αρχικά, η στήλη DATE OCC μετατρέπεται σε timestamp από τον οποίο εξάγονται έτος και μήνας. Στη συνέχεια, γίνεται ομαδοποίηση και μέτρηση των εγκλημάτων (`crime_total`) ανά (`year`, `month`). Ακολουθεί η χρήση *window function* κατά την οποία ορίζεται παράθυρο που χωρίζει τα δεδομένα ανά έτος και τα ταξινομεί φθίνουσα σειρά βάσει `crime_total`. Στη συνέχεια, εφαρμόζεται η συνάρτηση `rank()` πάνω σε αυτό το window partition, αποδίδοντας κατάταξη σε κάθε μήνα εντός του έτους. Τέλος, φιλτράρονται οι εγγραφές με `ranking ≤ 3` και το αποτέλεσμα ταξινομείται κατά έτος και κατάταξη.

```
1 ts = to_timestamp(col("DATE OCC"), "yyyy MMM dd hh:mm:ss a")
2 date_df = (crimes_df
3     .withColumn("year", year(ts))
4     .withColumn("month", month(ts)))
5
6 monthly_counts = (date_df
7     .groupBy("year", "month")
8     .agg(count("*").alias("crime_total")))
9
10 window = Window.partitionBy("year").orderBy(col("crime_total").desc())
11 ranked = monthly_counts.withColumn("ranking", rank().over(window))
12
13 result = (
14     ranked
15     .filter(col("ranking") <= 3)
16     .orderBy(col("year").asc(), col("ranking").asc())
17     .select("year", "month", "crime_total", "ranking")
18 )
```

Κώδικας 5: DFQ2.py - Window function rank() ανά έτος

3.2 Υλοποίηση SQL - SQLQ2.py

Τα CSV αρχεία φορτώνονται και καταχωρούνται ως προσωρινό view (`createOrReplaceTempView`). Η λογική είναι ταυτόσημη με την DataFrame υλοποίηση, εκφρασμένη σε SQL: το εσωτερικό subquery υπολογίζει το `crime_total` ανά (`year`, `month`), το μεσαίο εφαρμόζει `RANK() OVER (PARTITION BY year)` και το εξωτερικό φιλτράρει `ranking ≤ 3` και ταξινομεί το αποτέλεσμα.

```
1 # Register the DataFrame as a SQL view so it can be queried by name
2 crimes_df.createOrReplaceTempView("crimes")
3
4 results = spark.sql("""
5     SELECT year, month, crime_total, ranking
```

```

6 FROM (
7     SELECT year, month, crime_total,
8           RANK() OVER (
9               PARTITION BY year
10              ORDER BY crime_total DESC
11            ) AS ranking
12 FROM (
13     SELECT
14         YEAR(TO_TIMESTAMP('DATE OCC', 'yyyy MMM dd hh:mm:ss a')) AS
15         year,
16         MONTH(TO_TIMESTAMP('DATE OCC', 'yyyy MMM dd hh:mm:ss a')) AS
17         month,
18         COUNT(*) AS crime_total
19     FROM crimes
20     GROUP BY year, month
21 )
22 WHERE ranking <= 3
23 ORDER BY year ASC, ranking ASC
24 """

```

Κώδικας 6: SQLQ2.py - Pure SQL με RANK() OVER PARTITION BY

3.3 Σύγκριση Επίδοσης (4 executors × 1 core × 2 GB)

Πίνακας 4: Χρόνοι εκτέλεσης Query 2 - 3 εκτελέσεις, 4 executors × 1 core × 2 GB

Υλοποίηση	Run 1 (ς)	Run 2 (s)	Run 3 (s)	Median (s)
DataFrame	51.957	65.050	70.834	65.050
SQL	71.424	105.477	58.276	71.424

Σχολιασμός: Αμφότερες οι υλοποιήσεις παράγουν ίδια logical plans μέσω του Catalyst optimizer, διότι το DataFrame API μεταφράζεται εσωτερικά στον ίδιο σχεδιασμό με την αντίστοιχη SQL. Ως αποτέλεσμα, οι χρόνοι (65.1s έναντι 71.4s) είναι πολύ κοντά, με μικρή υπεροχή DataFrame που μπορεί να αποδοθεί σε run-to-run variance παρά σε δομική διαφορά.

4 Query 3 - DataFrame και RDD API

Το Query 3 υπολογίζει, ανά ZIP Code του Los Angeles, το μέσο ετήσιο κατακεφαλήν εισόδημα για τη διετία 2020–2021:

$$\text{per_capita_income} = \frac{\text{median_income} \times \text{households}}{\text{population}}$$

Τα δεδομένα πληθυσμού/νοικοκυριών προέρχονται από την απογραφή 2020 (GeoJSON), ενώ τα δεδομένα εισοδήματος από το CSV 2021.

4.1 Υλοποίηση DataFrame - DFQ3.py

Το GeoJSON είναι μια ενιαία δομή FeatureCollection, οπότε διαβάζεται με `multiLine=true`, ο πίνακας `features` «ανοίγει» με `explode` σε μία γραμμή ανά block και τα εμφωλευμένα πεδία του `properties` γίνονται flattened σε στήλες. Στη συνέχεια, γίνεται ομαδοποίηση ανά ZIP code με άθροιση πληθυσμού και νοικοκυριών, κρατώντας μόνο τις περιοχές με θετικό πληθυσμό (ώστε να αποφευχθεί διαίρεση με το μηδέν). Έπειτα, διαβάζεται το `income CSV`, στο οποίο το πεδίο εισοδήματος έχει μορφή "\$52,806", οπότε αφαιρούνται τα σύμβολα \$ και , με `regexp_replace`

πριν τη μετατροπή σε double. Τα δύο DataFrames ενώνονται (equi-join) στο κοινό ZIP code key⁴ και υπολογίζεται το κατακεφαλήν εισόδημα. Τέλος, επιλέγονται οι στήλες zipcode και per_capita_income και το αποτέλεσμα ταξινομείται κατά ZIP code.

```
1 # Explode the GeoJSON FeatureCollection
2 census_blocks_raw = (
3     spark.read.option("multiLine", "true").json(CENSUS_BLOCKS)
4     .selectExpr("explode(features) as features")
5     .select("features.*")
6 )
7 prop_fields = census_blocks_raw.schema["properties"].dataType.fieldNames()
8 flattened_blocks = census_blocks_raw.select([
9     col(f"properties.{c}").alias(c) for c in prop_fields
10 ])
11
12 # Sum population & households per ZIP code
13 zip_stats_df = (
14     flattened_blocks.groupBy(ZIPCODE_COL)
15     .agg(sum(col(POP_COL).cast("long")).alias("population"),
16         sum(col(HH_COL).cast("long")).alias("households"))
17     .filter(col("population") > 0)
18 )
19
20 # Clean format "$52,806" -> 52806.0
21 income_df = (
22     spark.read.csv(INCOME_CSV, header=True, sep=";")
23     .withColumn("median_income",
24         regexp_replace(col(INCOME_COL), "$[,]", "").cast("double"))
25     .filter(col("median_income").isNotNull())
26 )
27
28 # Join & compute per-capita income
29 results = (
30     join_zip_income(zip_stats_df, income_df, strategy)
31     .withColumn("per_capita_income",
32         round(col("median_income") * col("households") / col("population"),
33             2))
34     .select(col(ZIPCODE_COL).alias("zipcode"), "per_capita_income")
35     .orderBy("zipcode")
36 )
```

Κώδικας 7: DFQ3.py - Dataframe aggregation and equi-join ανά ZIP code

4.2 Υλοποίηση RDD - RddQ3.py

Η λογική στηρίζεται σε δύο RDDs με κοινό κλειδί το ZIP code, τα οποία ενώνονται με join. Το GeoJSON δεν μπορεί να διαβαστεί απευθείας με `sc.textFile` (είναι multiLine), οπότε φορτώνεται και γίνεται flattened μέσω DataFrame όπως πριν και μετατρέπεται σε RDD με `.rdd`. Πάνω σε αυτό εφαρμόζεται αλυσίδα transformations:

1. `map`: κάθε γραμμή μετατρέπεται σε ζεύγος (zipcode, (population, households)).
2. `filter`: αφαιρούνται οι εγγραφές με κενό ZIP code.
3. `reduceByKey`: αθροίζονται πληθυσμός και νοικοκυριά ανά ZIP code.
4. `filter`: απορρίπτονται οι περιοχές με μηδενικό πληθυσμό.

Το δεύτερο RDD προκύπτει από το CSV εισοδημάτων, που διαβάζεται με `textFile`. Όπως και στην Ενότητα 2, διαβάζεται πρώτα η επικεφαλίδα με `first()` ώστε να εντοπιστούν δυναμικά τα indexes των στηλών Zip Code και Estimated Median Income. Έπειτα:

⁴Ο Catalyst εδώ επιλέγει αυτόματα στρατηγική join, αλλά μέσω του argument `--join-strategy` μπορεί να επιβληθεί συγκεκριμένη (π.χ. broadcast), όπως εξετάζεται στην Ενότητα 6.

1. filter: αφαιρείται η γραμμή επικεφαλίδας.
2. map με csv.reader: κάθε γραμμή αναλύεται σε λίστα πεδίων.
3. map με parse_income: εξάγεται ζεύγος (zipcode, median_income), αφαιρώντας τα σύμβολα \$ και , από το εισόδημα.
4. filter: κρατούνται μόνο οι εγγραφές με θετικό εισόδημα.

Τέλος, τα δύο RDDs συνδυάζονται και υπολογίζεται το αποτέλεσμα:

1. join: τα δύο RDDs ενώνονται στο ZIP code, δίνοντας (zipcode, ((population, households), median_income)).
2. map: υπολογίζεται το $\text{per_capita_income} = \text{median_income} \times \text{households} / \text{population}$.
3. sortBy: ταξινομήση κατά ZIP code.
4. collect (action): τα αποτελέσματα επιστρέφονται στον driver.

```

1 def parse_income(parts, zip_idx, income_idx):
2     try:
3         median_income = parts[income_idx].strip().replace(",", "").replace("$", "")
4         return (parts[zip_idx].strip(), float(median_income))
5     except (ValueError, IndexError):
6         return (parts[zip_idx].strip(), 0.0)

```

Κώδικας 8: RddQ3.py - parse_income: καθαρισμός και εξαγωγή εισοδήματος

```

1 # GeoJSON -> RDD (zipcode, (population, households))
2 census_rdd = (
3     flattened_blocks.select(ZIPCODE_COL, POP_COL, HH_COL).rdd
4     .map(lambda row: (str(row[ZIPCODE_COL] or ""),
5                       (int(row[POP_COL] or 0), int(row[HH_COL] or 0))))
6     .filter(lambda kv: kv[0] != "")
7     .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
8     .filter(lambda kv: kv[1][0] > 0)
9 )
10
11 # income CSV -> RDD (zipcode, median_income)
12 income_rdd = (income_raw
13     .filter(lambda line: line != income_header)
14     .map(lambda line: next(csv.reader(io.StringIO(line), delimiter=";")))
15     .map(lambda p: parse_income(p, zip_idx, income_idx))
16     .filter(lambda kv: kv[1] > 0)
17 )
18
19 # Join & compute per ZIP
20 results = (
21     census_rdd.join(income_rdd)
22     .map(lambda kv: (kv[0],
23                     round(kv[1][1] * kv[1][0][1] / kv[1][0][0], 2)))
24     .sortBy(lambda kv: kv[0])
25     .collect()
26 )

```

Κώδικας 9: RddQ3.py - RDD aggregation and equi-join ανά ZIP code

4.3 Σύγκριση Επίδοσης (3 executors × 1 core × 2 GB)

Πίνακας 5: Χρόνοι εκτέλεσης Query 3 - 3 εκτελέσεις, 3 executors × 1 core × 2 GB

Υλοποίηση	Run 1 (s)	Run 2 (s)	Run 3 (s)	Median (s)
DataFrame	42.152	39.129	41.607	41.607
RDD	32.717	29.875	33.379	32.717

Σχολιασμός: Εδώ το RDD είναι ταχύτερο (διάμεσος 32.7s έναντι 41.6s), αντίστροφα από την Ενότητα 2. Αυτό οφείλεται στο μέγεθος των δεδομένων, καθώς το Query 3 επεξεργάζεται 91.626 census blocks και 282 εγγραφές εισοδήματος, τα οποία είναι πολύ λιγότερα από τα περίπου 3,1 εκατομμύρια εγκλήματα του Query 1. Στο συγκεκριμένο όγκο δεδομένων του ερωτήματος, οι αυτόματες βελτιστοποιήσεις του Catalyst δείχνουν να μην προλαβαίνουν να αποσβέσουν το κόστος σχηματισμού του query plan, ενώ η RDD pipeline εκτελεί απλώς ένα `reduceByKey` και ένα `join`. Η διακύμανση είναι χαμηλή και στις δύο υλοποιήσεις, γεγονός που υποδηλώνει σταθερή συμπεριφορά των αποτελεσμάτων.

5 Query 4 - Cross Join and Scalability

Το Query 4 υπολογίζει, ανά αστυνομικό τμήμα, τον αριθμό εγκλημάτων που τελέστηκαν πλησιέστερα σε αυτό (απ' οποιοδήποτε άλλο) και τη μέση απόστασή τους από το ίδιο. Χρησιμοποιήθηκε η *Haversine formula*⁵ για great-circle distance σε km, εκμεταλλευόμενοι τα latitude/longitude points των εγκλημάτων και των 21 σταθμών της LAPD.

5.1 Υλοποίηση DataFrame - DFQ4.py

Ξεκινάμε διαβάζοντας τα δύο CSV εγκλημάτων και κρατώντας μόνο το αναγνωριστικό (DR_NO) και τις συντεταγμένες LAT/LON, αφού απορριφθούν εγγραφές με κενές ή μηδενικές συντεταγμένες. Μετά, διαβάζεται το CSV των 21 σταθμών της LAPD και κρατούνται μόνο τα divisions και οι συντεταγμένες τους. Στη συνέχεια, μέσω του cross-join⁶ κάθε έγκλημα συνδυάζεται με καθέναν από τους 21 σταθμούς και για κάθε ζεύγος υπολογίζεται η απόσταση Haversine. Για κάθε έγκλημα εντοπίζεται ο πλησιέστερος σταθμός με single-pass argmin⁷ αντί για ταξινόμηση, εφαρμόζεται min πάνω σε ένα `struct(distance_km, division)`. Η σύγκριση struct είναι λεξικογραφική, οπότε το πρώτο πεδίο (distance_km) καθορίζει το ελάχιστο και «παρασύρει» μαζί του το division του πλησιέστερου σταθμού, χωρίς δεύτερο πέρασμα. Τέλος, γίνεται ομαδοποίηση ανά σταθμό με μέτρηση των εγκλημάτων που του αντιστοιχούν και τη μέση απόστασή τους, ταξινομημένα κατά φθίνουσα σειρά πλήθους.

```
1 def haversine_km(lat1, lon1, lat2, lon2):
2     dlat = F.radians(lat2 - lat1)
3     dlon = F.radians(lon2 - lon1)
4     a = (F.sin(dlat/2)**2
5         + F.cos(F.radians(lat1)) * F.cos(F.radians(lat2)) * F.sin(dlon/2)
6         **2)
7     return F.lit(6371.0) * 2 * F.atan2(F.sqrt(a), F.sqrt(1 - a))
```

Κώδικας 10: DFQ4.py - haversine_km: great-circle distance in km μεταξύ δύο latitude/longitude points

⁵<https://www.movable-type.co.uk/scripts/latlong.html>

⁶Ο Catalyst επιλέγει αυτόματα στρατηγική για τον cross join (εδώ BroadcastNestedLoopJoin): μέσω της παραμέτρου `--join-strategy` μπορεί να επιβληθεί συγκεκριμένη, όπως αναλύεται στην επόμενη ενότητα.

```

1 # Cross join: each crime x each station (21 stations)
2 dist_pairs_df = cross_join(crimes_df, stations_df, strategy).withColumn(
3     "distance_km",
4     haversine_km(F.col("LAT"), F.col("LON"), F.col("st_lat"), F.col("st_lon"
5         )))
6 # Single-pass argmin: struct comparison (distance_km, division)
7 min_dist_df = (
8     dist_pairs_df.groupBy(CRIME_ID)
9     .agg(F.min(F.struct("distance_km", "division")).alias("nearest"))
10    .select("nearest.division", "nearest.distance_km")
11 )
12
13 # Aggregate per station
14 results = (
15     min_dist_df.groupBy("division")
16     .agg(F.round(F.avg("distance_km"), 3).alias("average_distance"),
17         F.count("*").alias("#"))
18     .orderBy(F.col("#").desc())
19 )

```

Κώδικας 11: DFQ4.py - Cross join, argmin ανά έγκλημα και aggregation ανά police division

5.2 Στρατηγικές Join από τον Optimizer

Ο Catalyst επιλέγει **BroadcastNestedLoopJoin** για τον cross join, για τους εξής λόγους:

- **Non-equi join:** δεν υπάρχει equi-key condition, οπότε SortMergeJoin/ShuffledHashJoin αποκλείονται και απομένουν μόνο οι nested-loop στρατηγικές: Cartesian product και BroadcastNestedLoopJoin.
- **Μικρός πίνακας σταθμών:** το LA_Police_Stations.csv (21 εγγραφές) είναι ελάχιστο σε σχέση με τα περίπου 3,1 εκατομμύρια εγκλήματα, οπότε γίνεται broadcast σε κάθε partition των crimes και εκτελείται nested loop τοπικά, χωρίς shuffle.

5.3 Μελέτη Κλιμακωσιμότητας

Εξετάζονται δύο σενάρια κλιμάκωσης: η **A**-κάθετη (σταθεροί 2 executors, αύξηση cores/μνήμης) και η **B**-οριζόντια (σταθερός προϋπολογισμός 8 cores/16 GB, μεταβολή του πλήθους executors). Τα αποτελέσματα συνοψίζονται στους Πίνακες 6 και 7.

Σχολιασμός: Στο Study A (κάθετη κλιμάκωση) ο χρόνος μειώνεται μονότονα καθώς αυξάνονται οι πόροι ανά executor: 159.0s > 107.9s > 81.7s. Δηλαδή συμβαίνει σχεδόν υποδιπλασιασμός από 1 σε 4 cores. Αυτό είναι αναμενόμενο, αφού ο cross join με τον υπολογισμό Haversine είναι CPU-bound και επομένως περισσότερα cores ανά executor εκτελούν περισσότερους υπολογισμούς αποστάσεων παράλληλα.

Στο Study B (οριζόντια κλιμάκωση) η ταχύτερη διαμόρφωση είναι οι λίγοι «παχείς» executors (2×4 cores → 81.7s) έναντι των πολλών «λεπτών» (4×2 → 106.7s, 8×1 → 102.3s). Παρότι το σύνολο cores είναι ίδιο, η διάσπαση των πόρων σε περισσότερους executors επιβαρύνει, αφού θεωρητικά ο BroadcastNestedLoopJoin αντιγράφει τους 21 σταθμούς σε κάθε executor και προστίθεται overhead scheduling ανά pod, χωρίς όφελος για μια CPU-bound εργασία.

Πίνακας 6: Study A: Κάθετη κλιμακωσιμότητα Query 4 (σταθεροί 2 executors)

Executors	Cores	Memory(GB)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Median (s)
2	1	2	149.809	159.033	180.581	159.033
2	2	4	128.816	107.870	102.930	107.870
2	4	8	66.996	82.438	81.667	81.667

Πίνακας 7: Study B: Οριζόντια κλιμακωσιμότητα Query 4 (σταθερό σύνολο 8 cores/16 GB)

Executors	Cores	Memory(GB)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Median (s)
2	4	8	66.996	82.438	81.667	81.667
4	2	4	82.208	108.710	106.687	106.687
8	1	2	97.864	102.262	116.266	102.262

6 Join Strategies - Hint & Explain

Για τον εντοπισμό και την πειραματική σύγκριση στρατηγικών join, χρησιμοποιούνται:

- `.hint("strategy")` ή `broadcast(df)`: προτρέπει τον Catalyst να επιλέξει συγκεκριμένη στρατηγική.
- `.explain("formatted")`: εκτυπώνει το physical plan για επαλήθευση.

6.1 Query 3 - DataFrame API

Η υλοποίηση υποστηρίζει επιλογή στρατηγικής μέσω ορίσματος `--join-strategy`:

```

1 def join_zip_income(zip_stats_df, income_df, strategy: str):
2     cond = col(ZIPCODE_COL) == col(INCOME_ZIP_COL)
3     if strategy == "default":
4         return zip_stats_df.join(income_df, cond)
5     if strategy == "broadcast":
6         return zip_stats_df.join(broadcast(income_df), cond)
7     # merge / shuffle_hash / shuffle_replicate_nl
8     return zip_stats_df.join(income_df.hint(strategy), cond)
9
10 # Verify physical plan
11 results.explain("formatted")
12 # Example output (default / broadcast):
13 #   BroadcastHashJoin, BuildRight

```

Κώδικας 12: DFQ3.py — Εφαρμογή διαφορετικών στρατηγικών join

6.2 Query 4

Για το Query 4, λόγω cross join χωρίς equi-key, μόνο nested-loop στρατηγικές εφαρμόζονται:

```

1 def cross_join(crimes_df, stations_df, strategy: str):
2     if strategy == "default":
3         return crimes_df.join(stations_df)
4     if strategy == "broadcast":
5         return crimes_df.join(F.broadcast(stations_df))
6     # shuffle_replicate_nl: CartesianProduct
7     # merge / shuffle_hash: ignored by Catalyst (no equi-key)
8     #                                     -> fallback to BroadcastNestedLoopJoin
9     return crimes_df.join(stations_df.hint(strategy))

```

6.3 Σύγκριση Στρατηγικών

Πίνακας 8: Query 3 (equi-join): physical plan και χρόνοι ανά hint

Hint	Physical Plan	Run 1 (s)	Run 2 (s)	Run 3 (s)	Median (s)
<code>broadcast</code>	BroadcastHashJoin	38.133	38.593	41.929	38.593
<code>merge</code>	SortMergeJoin	40.693	39.175	41.255	40.693
<code>shuffle_hash</code>	ShuffledHashJoin	40.172	40.611	47.571	40.611
<code>shuffle_replicate_nl</code>	CartesianProduct	45.533	57.677	44.195	45.533

Πίνακας 9: Query 4 (non-equi cross join): physical plan και χρόνοι ανά hint

Hint	Physical Plan	Run 1 (s)	Run 2 (s)	Run 3 (s)	Median (s)
<code>broadcast</code>	BroadcastNestedLoopJoin	109.402	100.868	80.963	100.868
<code>shuffle_replicate_nl</code>	CartesianProduct	112.977	76.836	97.399	97.399

Στο Query 4 τα `merge/shuffle_hash` ανάγονται στο ίδιο physical plan με το `broadcast` (BroadcastNestedLoopJoin), οπότε δεν μετρήθηκαν ξεχωριστά.

Σχολιασμός: Στο Query 3 (equi-join) όλες οι στρατηγικές αποδίδουν κοντά, με ταχύτερη τη BroadcastHashJoin. Λόγω του μικρού μεγέθους του πίνακα `income_df`, γίνεται broadcast χωρίς shuffle και η αντιστοίχιση γίνεται με hash lookup. Οι SortMergeJoin/ShuffledHashJoin είναι ελαφρώς πιο αργές (εισάγουν shuffle), ενώ η `shuffle_replicate_nl` εξαναγκάζει CartesianProduct (πλήρες καρτεσιανό γινόμενο με το equi-condition ως φίλτρο) και είναι η πιο αργή. Οι διαφορές παραμένουν μικρές λόγω του μικρού μεγέθους των δεδομένων.

Στο Query 4 (non-equi cross join) εφαρμόζονται μόνο nested-loop στρατηγικές.

Τα `merge/shuffle_hash` αγνοούνται και ανάγονται σε BroadcastNestedLoopJoin. Λόγω του μικρού μεγέθους του πίνακα `stations_df` (21 σταθμοί), η διαφορά είναι μικρή ανάμεσα στις στρατηγικές CartesianProduct (97.4s) και BroadcastNestedLoopJoin (100.9s). Και οι δύο είναι nested-loop στρατηγικές και διαφέρουν στον τρόπο που φέρνουν τη μικρή πλευρά κοντά στα crimes: η BNLJ κάνει broadcast τους σταθμούς μία φορά σε κάθε executor, ενώ ο CartesianProduct τους αναπαράγει ζευγαρώνοντας τα partitions. Επειδή οι σταθμοί χωρούν σε ένα μόνο partition, το κόστος αυτής της μετακίνησης είναι αμελητέο και στις δύο περιπτώσεις, οπότε η BNLJ δεν έχει σημαντικό πλεονέκτημα.